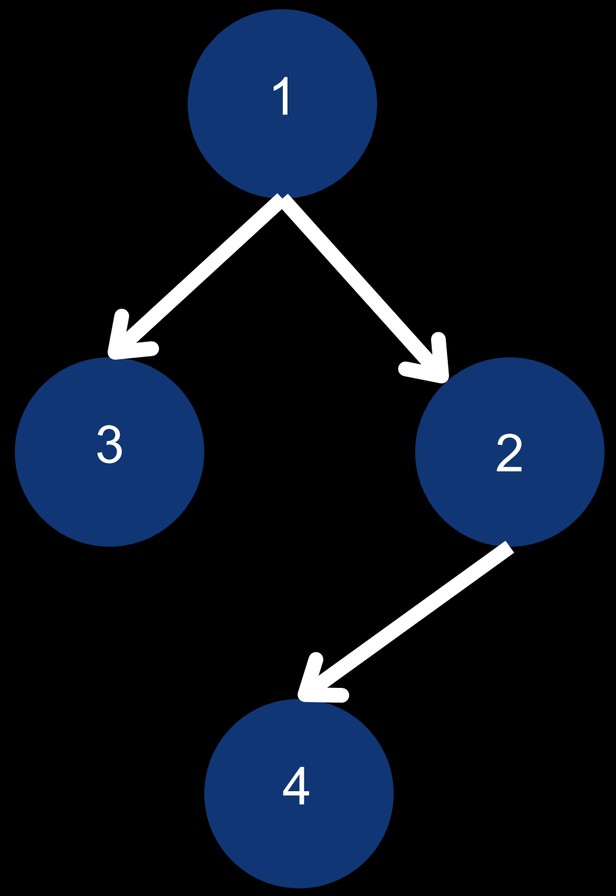
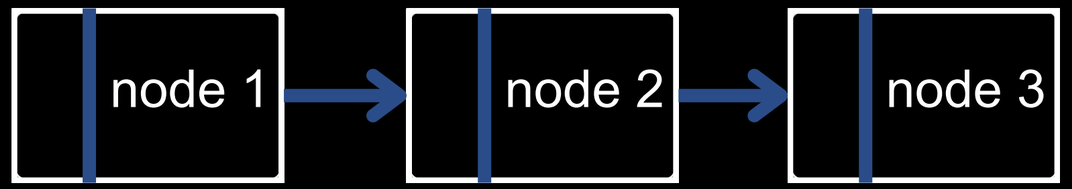
DATA STRUCTURE





essammohamedst1@gmail.com

Quadratic Sorting algorithms

Bubble Sort

Bubble Sort is a foundational sorting method. Its goal is simple: move the largest values to the end of a list, one step at a time, until everything is in order.

How it Works: The "Step-by-Step"

Compare Pairs: It looks at the first two numbers. If the left one is larger, they swap.

Slide Right: It moves one position to the right and compares the next two numbers.

The "Bubble" Effect: By the end of the first pass, the largest number has "bubbled" all the way to the final position.

Shrink the Workload: Since the last number is now in its correct spot, we ignore it and repeat the process for the remaining numbers.

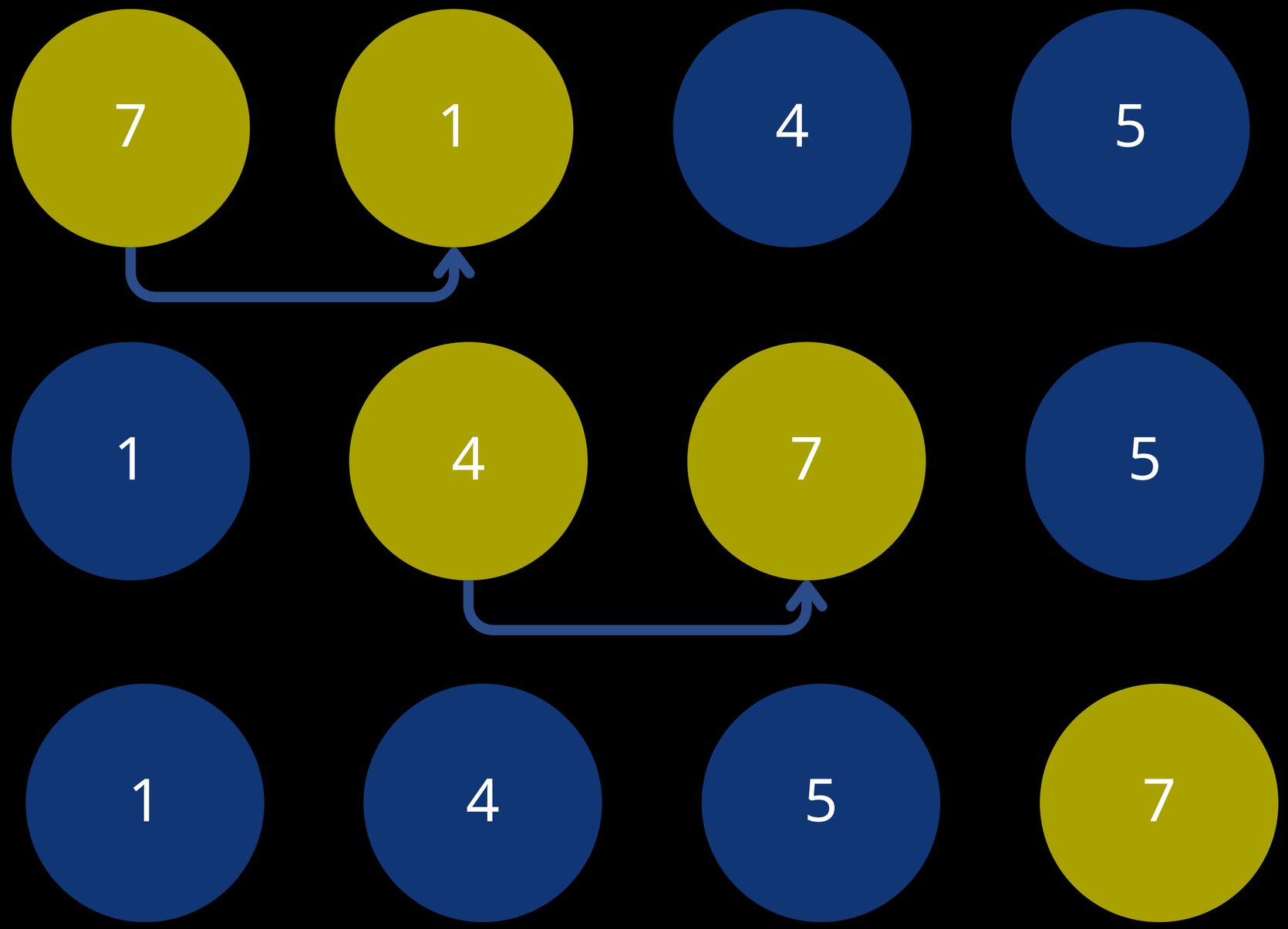
Pseudocode

Create two loops. The top loop should run for as many elements as there are in the array. The nested loop should run until the end of the array the first time, but each time after, should decrease by 1 as the end of the array is already sorted.

Starting at the beginning of the array, compare each element to the one next to it. If the value on the left is greater than the value on the right, swap it.

Repeat this process until the top loop finishes.

Optimization: If a complete pass occurs where no swaps happen, the array is already sorted — return the array.


7
1
4
5
1
4
7
5
1
4
5
7

Selection Sort

Selection Sort is a "look-ahead" algorithm. Instead of swapping constantly like Bubble Sort, it waits until it finds the absolute smallest value before making a move.

The Step-by-Step Logic

Set the "Starting Line": Start at the first position (Index 0).

Scan the Horizon: Look through every other number in the remaining list to find the smallest one.

The Swap: Once you've found the winner, swap it with the number at your "Starting Line."

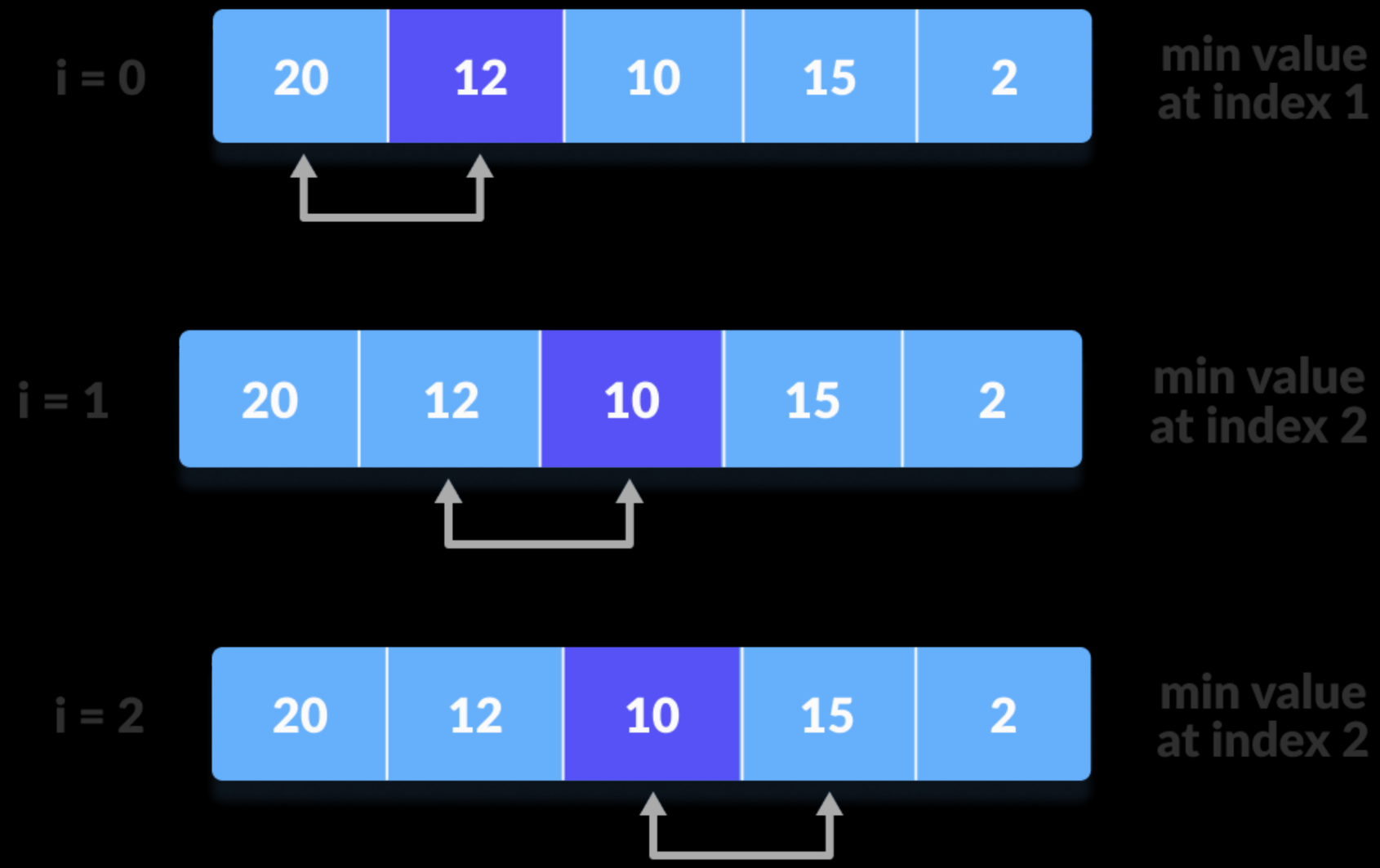
Why is it called "Selection"?
Because at every step, you are selecting the best candidate (the smallest number) and placing it exactly where it belongs.

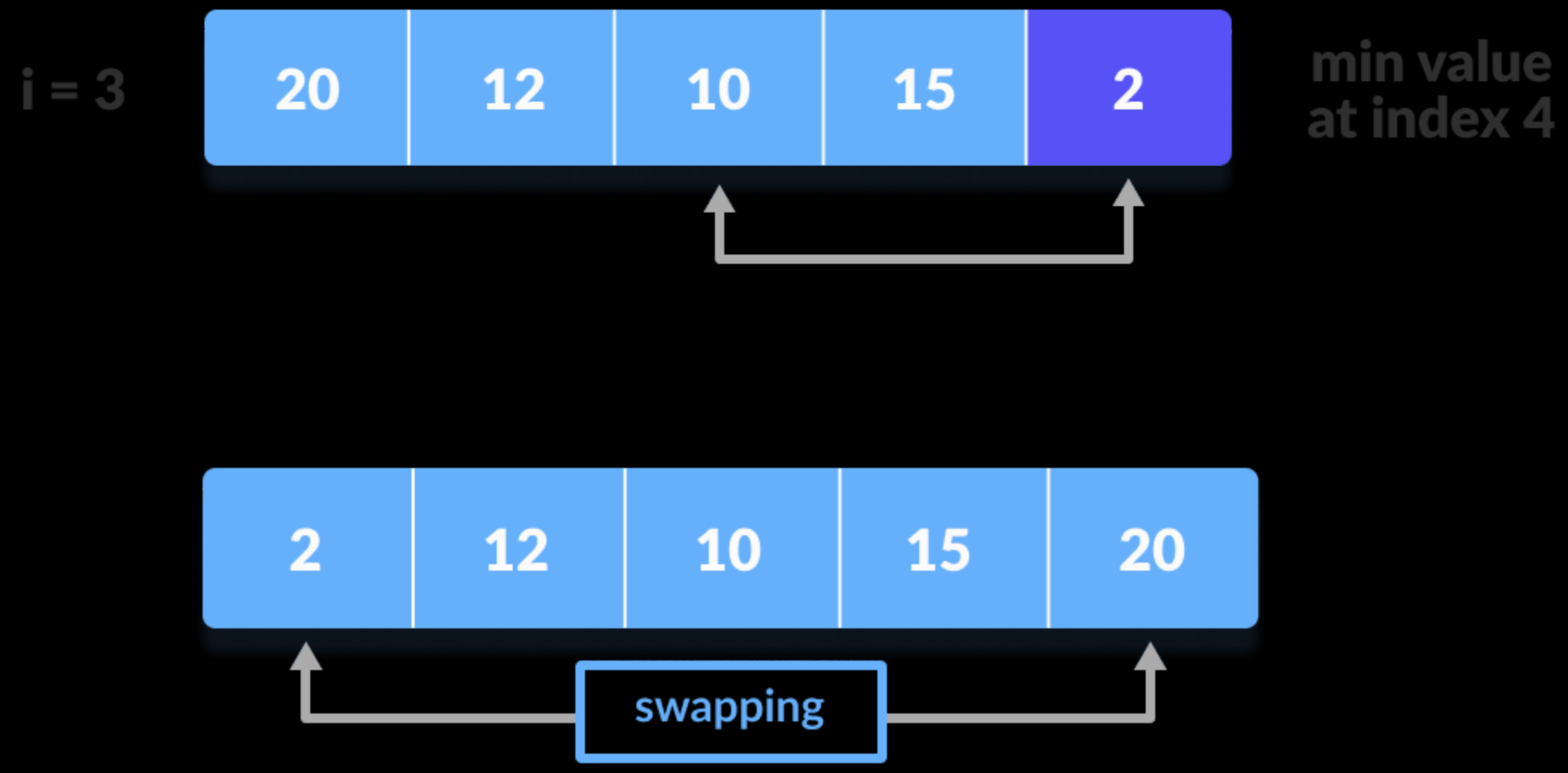
Pseudocode

Create two loops — one nested inside the other. The top loop progresses the array while the nested loop checks that value against all the remaining values.

Create a variable called minimum. Compare each element to the minimum. If it is smaller, save that value’s index as the new minimum.

After the nested loops iteration is complete, place the value located at the minimum index at the index of the top loop — unless the two values are the same.


i = 0
20
12
10
15
2
min value
at index 1
i = 1
20
12
10
15
2
min value
at index 2
i = 2
20
12
10
15
2
min value
at index 2


i = 3
20
12
10
15
2
min value
at index 4
2
12
10
15
20
swapping

Insertion Sort

Insertion Sort is an "active" sorter. It builds a sorted section of the list one item at a time by taking a new value and inserting it into its correct spot among the values it has already seen.

The Step-by-Step Logic

- The Starting Point: We assume the first item is already "sorted" by itself. We begin with the second value.
- The Pickup: We "pick up" this current value and compare it to the one immediately to its left.
- The Shift: If the value on the left is larger, we shift that larger value one spot to the right to m a gap.
- The Drop: We keep shifting until we find a number smaller than our current value (or reach the start of the list). Then, we drop our value into that empty gap.

Why is it called "Insertion"?
Because you aren't just swapping two neighbors—you are finding the perfect "hole" for a value and inserting it there.

Pseudocode

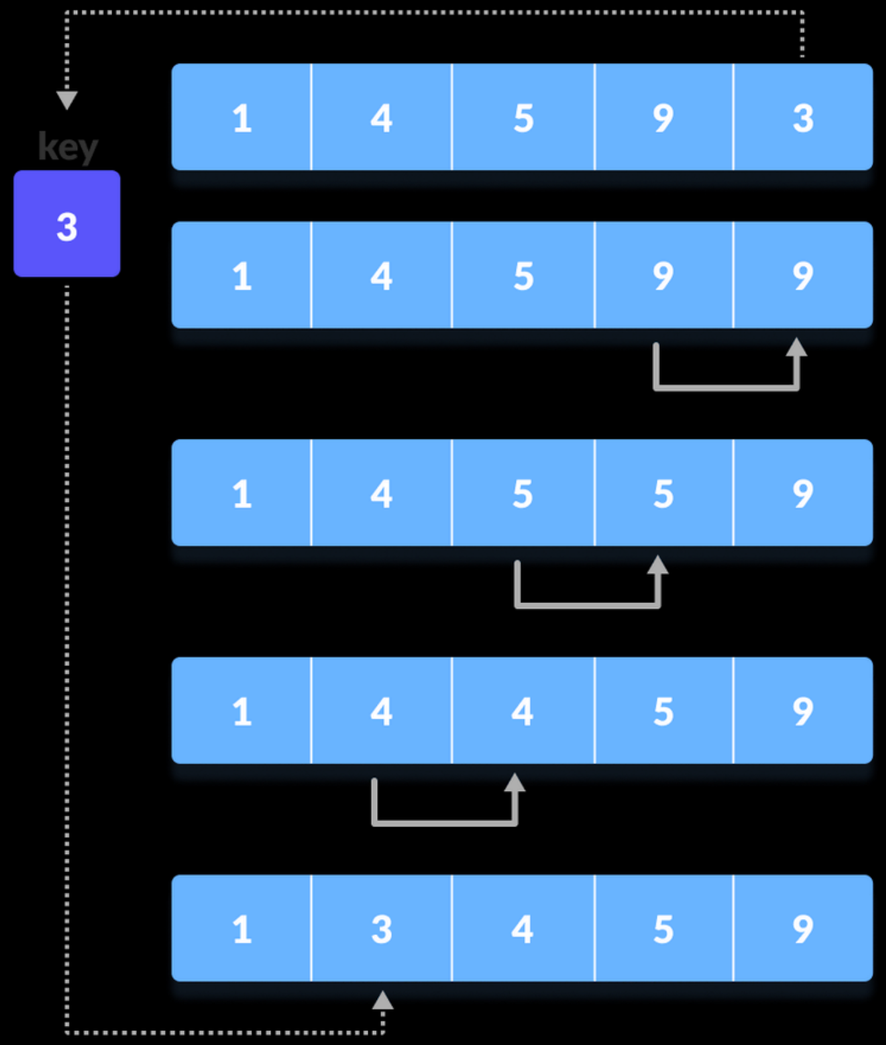
Create two loops — one nested inside the other. The top loop starts at the second value in the array. The nested loop starts one value less than the top loop’s index.

Create a variable called currentValue which is equal to the value located at the top loop’s index.

Starting from top loop’s index, work towards the beginning of the array, comparing the currentValue to each element. If the element is greater than the currentValue variable, copy the element’s value one index greater than its current position.

Repeat this process until you reach the beginning of the array or the currentValue is not greater than the valuing you are testing against.

Set element’s index to the current Value.


key
3
1 4 5 9 3
1 4 5 9 9
1 4 5 5 9
1 4 4 5 9
1 3 4 5 9

Algorithm Visualizer

Sorting (Bubble, Selection, Insertion, Merge, Quick, Counting, Radix)

Sorting is a very classic problem of reordering items (that can be compared, e.g., integers, floating-point numbers, strings, etc) of an array (or a list) in a certain order (increasing, non-decreasing (increasing or flat), decreasing, non-increasing…

visualgo.net

Thanks

All code and examples can be found on GitHub https://github.com/ESSAMMOHAMED1